Parcial Algoritmos y Programación II - 7541 Curso Calvo (2º instancia) 20 de noviembre de 2015

Padrón: **Apellido y Nombre:**

Punteros: APROBADO - DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){

  int *A, *C, *F;
  int **B;
  char *D, *E;
  int H = 66;
  char G = 'C';

  F = new int[3];

  for (int i = 0; i < 3; i++) {
      F[i] = H + i;
  }

  H++;
  A = F;
  C = new int;
  (*C) = A[1];
  A = F + 2;

  cout << (*F) << (*C) << A[0] << endl;

  B = &C;
  A = C;
  C = F + 1;
  (**B) = H + 2;

  cout <<  (*C) <<  (**B) << (*A) << endl;

  D = (char*) A;
  E = (char*) *B;
  F[1] = 70;

  cout << (*D) << (*E) << (*A) << endl;

  E = &G;
  H++;
  G = (char) H;
  H++;
  (*A)    = H;

  cout << (*E) << (*D) << G << endl;

  // liberar la memoria
  // ...

  return 0;
}
```

2. Implementar para la clase Lista<T> con una estructura **doblemente enlazada** el siguiente método, indicando pre y post condiciones:

```
void agregar(T elemento, unsigned int posicionElemento);
```

3. Implementar el método `buscarMesasEnLasQueFueronMuchoMasVotados` de la clase `ComiteElectoral` a partir de las siguientes especificaciones:

```
class ComiteElectoral {
  public:
    /* post: devuelve una nueva Lista con los números de Mesa en las que los Candidatos dados (EN CONJUNTO)
     *       recibieron más votos que el total de los demás candidatos en dicha Mesa.
     *       Solo considera Votos válidos (no recurridos) y que no son en blanco.
     */
    Lista<int>* buscarMesasEnLasQueFueronMuchoMasVotados(List<MesaElectoral*>* mesas, Lista<Candidato>* candidatos);
}
```

```
class MesaElectoral {
  public:
    /* post: Mesa identificada por 'número' y sin Votos.
     */
    MesaElectoral(int numero);

    /* post: devuelve el número que identifica la Mesa.
     */
    int obtenerNumero();

    /* post: devuelve todos los Votos (válidos + recurridos).
     */
    Lista<Voto*>* obtenerVotos();

    /* post: devuelve aquellos Votos que fueron recurridos.
     */
    Lista<Voto*>* obtenerVotosRecurridos();

    /* post: libera todos los Votos acumulados.
     */
    ~MesaElectoral();
};
```

```
class Voto {
  public:
    /* post: voto para el Candidato indicado.
     */
    Voto(Candidato candidatoVotado);

    /* post: voto en blanco.
     */
    Voto();

    bool esBlanco();

    /* pre : no es un voto en blanco.
     * post: devuelve el Candidato votado.
     */
    Candidato obtenerCandidato();
}
enum Candidato {

  CANDIDATO_1, CANDIDATO_2, CANDIDATO_3
  , …, CANDIDATO_N;
}
```

4. Diseñar la especificación e implementar un TDA que modele una **Montacarga**. Debe proveer operaciones para:

- crear el Montacargas recibiendo como parámetro el peso máximo (Kg) que puede cargar.
- obtenerCarga: devuelve el peso (Kg) de la carga que tiene actualmente.
- cargar: sube al montacargas una carga con el peso (Kg) indicado.
- trasladar: mueve el montacargas la distancia (m) indicada.
- descargar: baja del montacargas toda la carga que tiene y devuelve la distancia total que la trasladó.
- calcularCargaPromedio: devuelve el peso (Kg) promedio de las cargas completadas (cargadas, trasladadas y luego descargadas)

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas